

Universidad de Ingeniería y Tecnología

Departamento de Ingeniería Mecatrónica



Fundamentos de Robótica

Profesores: Oscar E. Ramos, Alexander López

Laboratorio 4

Movimiento del robot UR5

Lima - Perú

2026-1

Laboratorio 4

Movimiento del robot UR5

Objetivos

- Modelar, calcular y verificar la cinemática directa de un robot manipulador de seis grados de libertad (UR5) usando la convención de Denavit-Hartenberg estándar.
- Implementar cinemática directa para el robot UR5 con base en métodos numéricos.
- Leer los datos provenientes de los encoders del robot utilizando ROS.
- Realizar un enlace entre los sensores y el movimiento del robot UR5.

Equipo y Materiales

Descripción	Cantidad
Computadora con Internet	1

Indicaciones del Laboratorio

1. Todos los laboratorios son evaluados, por tanto se debe leer la guía del laboratorio antes de asistir a cada laboratorio.
2. El laboratorio comienza a más tardar 5 minutos después de la hora de inicio.
3. A los 5 minutos de iniciar el laboratorio se empieza con la prueba de entrada, la cual se debe desarrollar de manera personal. Esta prueba de entrada cuenta como parte de la nota del laboratorio, la cual evalúa que el alumno@ haya leído la guía de laboratorio. La prueba tiene una duración de 3 o 4 minutos, dependiendo la cantidad de preguntas.
4. Los alumnos que lleguen 20 minutos después de la hora inicio del laboratorio, son libres de asistir, pero no tendrán nota en dicho laboratorio.
5. El laboratorio se desarrolla en pareja de 2 personas (solo será de 3 personas en caso falte un alumno@ que no tenga pareja).
6. La prueba de salida será evaluada por medio de un cuestionario colgado en canvas, el cuestionario es llenado por cada uno de los integrantes del grupo, se puede realizar la prueba de manera grupal, pero cada alumno lo debe hacer desde su propio canvas.
7. La parte práctica será revisada por el profesor durante el laboratorio, es posible que se haga preguntas de los códigos desarrollados.
8. Está prohibido copiar, compartir información o conversar con otro grupo. Los alumnos involucrados serán presentados al comité de ética de la universidad.

Resumen del laboratorio

Se presenta un resumen de los puntos a tratar en el siguiente laboratorio.

1. Prueba de entrada.
2. Descargar los paquetes para este laboratorio.
3. Parámetros Denavit-Hartenberg.
4. Matriz de transformación.
5. Cinemática Directa.
6. Actividades.
7. Evaluación en Canvas.
8. Implementación.

1. Paquetes del laboratorio

Para trabajar en este laboratorio, es necesario descargar el repositorio del curso en el espacio de trabajo “lab_ws”:

```
$ cd # Ubicarse en la raíz del sistema
$ cd lab_ws/src
$ git clone https://github.com/utecrobotics/fundrobotica-20261 frlabs
```

En caso que el espacio de trabajo “lab_ws” haya sido borrado, el alumno@ debe volver a crearlo y hacer que el sistema lo reconozca como un espacio de trabajo de ROS2. También se debe clonar los repositorios (de github) que contienen toda la información y los drivers del robot “UR5”, tanto para simulación como para uso con el robot real. Para mantener orden en el espacio de trabajo, dichos repositorios serán clonados dentro de una carpeta llamada “UR5” que se encontrará en el espacio de trabajo (creado en el laboratorio 1). Los comandos a utilizar son los siguientes:

La ejecución de este nodo se realizada con la siguiente línea de código:

```
$ cd lab_ws/src
$ sudo apt-get install ros-humble-simple-actions ros-humble-ros2-control* ros-humble-moveit
```

Crear un nuevo workspace:

```
$ mkdir -p ur_ws/src
$ cd ur_ws/src
$ git clone -b humble https://github.com/UniversalRobots/Universal_Robots_ROS2_GZ_Simulation.git
$ git clone -b humble https://github.com/UniversalRobots/Universal_Robots_ROS2_Description.git
```

Para ambos casos, compilar los paquetes descargados:

```
$ cd ..  
$ colcon build  
$ source /home/<user>/ur_ws/install/setup.bash
```

2. Cinemática de Robots Manipuladores

El estudio de la cinemática directa de robots manipuladores industriales proporciona elementos para analizar y diseñar el desplazamiento de trayectorias del robot, así como la orientación del efector-final (herramienta colocada al final del brazo robot). Dependiendo del tipo de articulación que se encuentran incluidas en la estructura mecánica del robot (lineales o rotacionales), se presenta una clasificación general de robots manipuladores: antropomórfico, esférico, cilíndrico, scara y cartesiano. En términos más exactos, un brazo robótico es una cadena cinemática abierta, compuesta por varias articulaciones, las cuales se representan por los ejes de coordenadas presentados en la Figura 1.

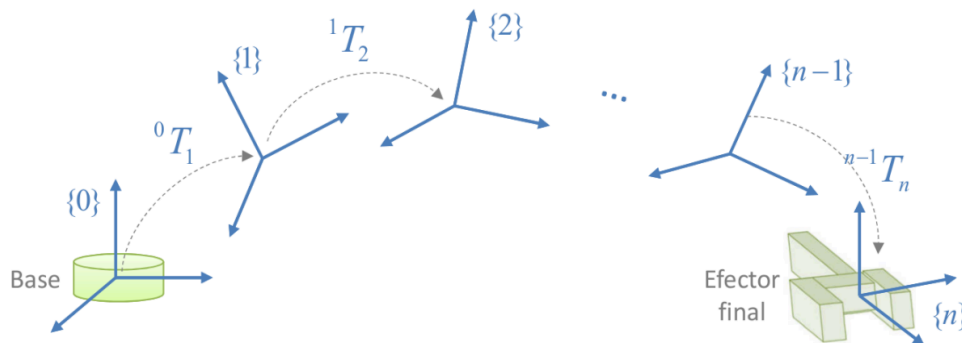


Figura 1: Cadena cinemática abierta.

El posicionamiento del robot (pose) en el espacio tridimensional requiere de 6 coordenadas: 3 coordenadas para su posición cartesiana y 3 coordenadas para la orientación del efector-final, la posición y orientación del efector se aprecia en la Figura 2, con el sistema X_2Y_2 . **En caso el brazo robótico tenga una herramienta, el extremo final de la herramienta se considera el efector-final.** Las herramientas pueden ser garras para sostener objetos, torchas de soldar, pulidores, etc. Es fundamental conocer la posición de las herramientas o efector-final, para saber si la herramienta de trabajo está en la posición y orientación requerida. Sin embargo, solo se tiene el control de la posición angular de las articulaciones. Por tanto, se denominó a la relación entre las posiciones articulares de los actuadores del robot con la posición y orientación del

efector final como la cinemática directa, una representación sencilla se puede observar en la Figura 2.

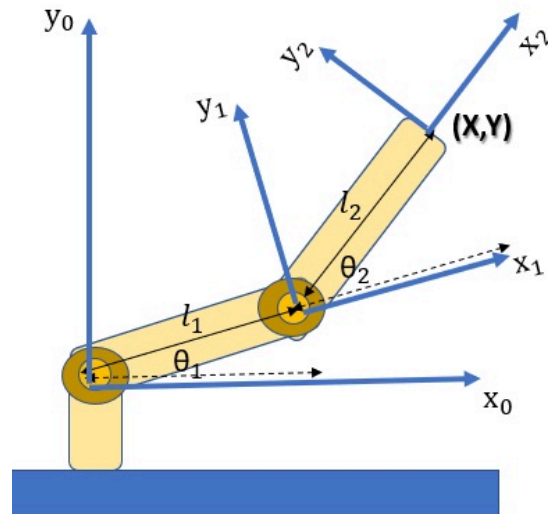


Figura 2: θ_1 y θ_2 representan las posiciones articulares, mientras X_2 y Y_2 indican la posición y orientación del efector final con respecto a X_0 y Y_0 .

La cinemática directa se puede hallar a través de aplicación de la transformación homogénea a cada una de las articulaciones, hasta llegar al efector-final.

2.1. Transformaciones homogéneas

La representación de posicionamiento para robots manipuladores involucra sistemas de coordenadas cartesianos que especifican posición y orientación de cada uno de los actuadores hasta el efector-final del robot. **La transformación homogénea es una herramienta matemática que involucra operaciones de rotación y traslación dentro de una matriz que estructura el modelo de cinemática directa.** En el caso de un brazo robótico, se establece un sistema de coordenadas en la base y en cada una de las articulaciones como se mostró en la Figura 1.

La matriz homogénea permite encontrar la relación entre un sistema de coordenadas y su siguiente sistema de coordenadas dentro de la cadena cinemática. Esta relación se expresa a través de una matriz de 4x4 como se muestra en la Figura 3, donde cada columna está representada por un color (cada una de 3 filas), la “n” de color rojo, la “o” de color verde y la “a” de azul, cada una representa el cambio de rotación con respecto de un sistema a otro. La última columna “p” (de 3 filas) representa la traslación (el cambio de posición) con respecto de un sistema a otro. La última fila de ceros y un único uno se coloca para que la matriz sea cuadrada, esto es importante ya que en caso contrario no se podría determinar la inversa de la matriz.

$${}^0T_e = \begin{bmatrix} n_x & o_x & a_x & p_x \\ n_y & o_y & a_y & p_y \\ n_z & o_z & a_z & p_z \\ \hline 0 & 0 & 0 & 1 \end{bmatrix}$$

Figura 3: La transformación homogénea indica el cambio de posición y orientación con respecto a una articulación y la siguiente en una misma cadena cinemática.

La transformación homogénea se puede determinar hallando la matriz homogénea, esta se determina utilizando la notación Denavit-Hartenberg. Esta notación, enseñada en clase, se expresa a través de un conjunto de parámetros, longitudes y ángulos. La notación Denavit-Hartenberg permite obtener dichos parámetros y establecer de manera concreta la estructura matemática de las transformaciones homogéneas, de la cual se desprende el modelo cinemático directo.

$${}^{i-1}T_i(\theta_i, d_i, \alpha_i, a_i) = \begin{bmatrix} \cos \theta_i & -\cos \alpha_i \sin \theta_i & \sin \alpha_i \sin \theta_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \alpha_i \cos \theta_i & -\sin \alpha_i \cos \theta_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Figura 4: Transformación homogénea.

La transformación homogénea se realiza con respecto a una articulación a la siguiente articulación dentro de la cadena cinemática. Por ejemplo, como se muestra en la Figura 4, se puede calcular la transformación de “i” con respecto a “i-1”. Para calcular la matriz de transformación homogénea del efector final, con respecto a la base (o un punto inicial), se debe calcular la matriz para cada una de las articulaciones de la cadena cinemática abierta. Las matrices se deben multiplicar entre ellas usando una multiplicación tipo punto, como se muestra en la Figura 5.

$${}^0T_n = ({}^0T_1)({}^1T_2) \cdots ({}^{n-2}T_{n-1})({}^{n-1}T_n)$$

Figura 5: Transformación homogénea desde el sistema de referencia “0” hasta el sistema de referencia “n”.

3. Cinemática Directa del UR5

Para probar el modelo cinemático directo a partir de las medidas mostradas en la Figura 6, luego de haber determinado los parámetros de Denavit-Hartenberg, se debe realizar la siguiente actividad. Es importante mencionar que si quisiera ver el robot con sliders, puede visualizarlo en Rviz con el siguiente comando:

```
$ ros2 launch ur_description view_ur.launch.py ur_type:=ur5e
```

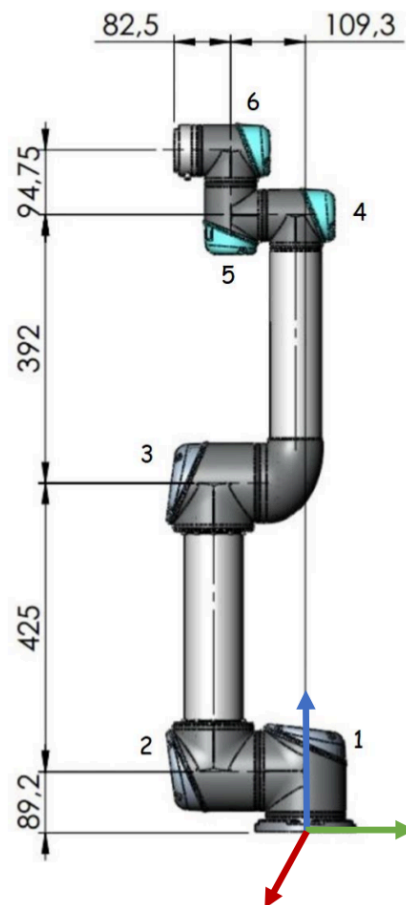


Figura 6: Robot UR5.

Para probar que el cálculo teórico de la cinemática directa, y su implementación práctica son coherentes, se utilizará el modelo del robot en el visualizador RViz. Con este fin, se ejecutará un nodo llamado “test_fkine”, el cual muestra al robot y a una pelota verde cuya posición está indicada por la función “fkine_ur5” realizada anteriormente.

La ejecución de este nodo se realiza con la siguiente línea de código:

```
$ ros2 launch lab4 display.py ur_type:=ur5e
```

Para ejecutar el nodo, usar la siguiente línea de código.

```
$ ros2 run lab4 test_fkine
```

Si la cinemática directa ha sido calculada correctamente, debe aparecer un eje (rojo para “x”, verde para “y”, azul para “z”) en la parte final de la cadena cinemática del robot. Cambiar otros valores del vector de configuración articular para observar diferentes posturas del robot UR5. Para cada una de estas configuraciones, verificar que el eje aún se encuentra en el efector-final.

Si es así, entonces no hay problemas con la cinemática directa calculada. Para verificar, se puede crear una línea recta en el espacio operacional (de la tarea), y se puede verificar que el eje siempre se encuentra al final de la cadena cinemática.

3. Simulación del UR5

3.1. Movimiento en Gazebo

Para realizar la prueba con el simulador, se utilizará el archivo de tipo launch llamado ur5.launch del paquete “ur_gazebo”, el cual abre el simulador gazebo y localiza al robot UR5 dentro de este simulador. Luego, se ejecutará un nodo de ejemplo, llamado “test_gazebo”, el cual modifica el valor de una articulación del robot UR5 y lo envía al simulador Gazebo. Para esto, ejecutar los dos siguientes comandos en diferentes terminales:

La ejecución de este nodo se realiza con la siguiente línea de código:

```
$ ros2 launch ur_simulation_gazebo ur_sim_control.launch.py
```

Para ejecutar el nodo, usar la siguiente línea de código:

```
$ ros2 run lab4 test_gazebo __ns:=joint_trajectory_controller
```

3.2. Conectarse a los sensores usando ROS

En este laboratorio se trabajarán con los nodos suscriptores, estos nodos reciben mensajes de los publicadores para realizar una tarea, para más información consultar al video en el siguiente [link](#).

Esta sección mostrará cómo se lee los datos del teclado usando ROS **como ejemplo**. Para poder usar el teclado en ROS, se debe ejecutar el nodo llamado “key_publisher”. En un terminal, ejecutar el siguiente comando (antes dar permisos de ejecución al nodo):

```
$ ros2 run lab4 key_publisher.py
```


Se debe tener en cuenta que el nodo “key_publisher” publica los datos en un tópico llamado “/keys”, el cual contiene mensajes de tipo “std_msgs/String”. Para ver los datos del teclado que están siendo publicados, se puede hacer un echo a este tópico (en otra nueva ventana de terminal, se deben poder visualizar ambos terminales al mismo tiempo) de la siguiente manera:

```
$ ros2 topic echo /keys
```

Inicialmente es probable que no aparezca nada, pero al presionar una tecla en el terminal donde se ejecuta el nodo “key_publisher”, se verá que aparece un mensaje con la tecla presionada. Se recomienda identificar a qué corresponde cada tecla, dado que esto será utilizado luego.

Nodo de ROS para leer datos del tópico: Una vez que se ha verificado que los datos del teclado son publicados en un tópico, es necesario acceder a los mismos. Para esto se debe crear un nodo que se suscriba al tópico usando la función “node.create_subscription”. Un ejemplo se muestra en el archivo “test_keyboard”, el cual puede ser ejecutado (en otra nueva ventana de terminal, se deben poder visualizar ambos terminales al mismo tiempo) a modo de prueba.

En este caso se tendrá que usar nodo que pueda publicar y suscribirse a ciertos tópicos.

4. Acciones en ROS

4.1. Definición

Las acciones están reservadas para procesos de cierta duración, una mayor al apagar o prender una lámpara. Las acciones tienen tres componentes (objetivo, retroalimentación y resultado); una acción tiene dos partes a programar, un servidor y un cliente. El objetivo es el elemento inicial para dicha acción, en los clientes se indica la acción a realizar, luego este se envía al servidor. El servidor puede devolver información al cliente del estado de la acción, realimentación. Por último, se devuelve un resultado, el cual puede o no ser satisfactorio.

4.2 Programación

Pasos para programar el servidor de una acción:

1. Definir el objetivo, retroalimentación y resultado.
2. Crear un paquete.
3. Agregar los componentes en el “package.xml” del paquete.
4. Agregar los componentes en el “CMakeLists.xml” del paquete.
5. Crear el nodo del paquete desde otro paquete.

Los pasos se explican en el siguiente [videotutorial](#).